PROG7311 Formative

Leo van Niekerk

ST10445055

GROUP 3

Part 1

Executive Summary

TechMove Logistics is currently relying on a mix of spreadsheets, emails, and phone calls to manage important operations like freight contracts, driver schedules, and invoicing. This multidimensional way of working has created a lot of problems / inconsistencies like missing documents, expired contracts going unnoticed, and delays in daily tasks which can hurt the business in many ways and is quite unacceptable. Honestly, this system is inefficient and not reliable enough for a growing international logistics company and with all the problems that they face , operations will never truly be optimal.

To solve these issues, a Global Logistics Management System will be developed as a web based application. The system will bring all major operations such as contract management, service requests, and financial processes into one place (no need to switch between multiple applications). It will also be designed to be reliable, scalable, and able to support the company as it expands.

As the Chief Solutions Architect, this report presents the technical approach before development begins. It explains the architecture framework that will guide the project, gives a high level view of the system design, introduces three important design patterns, and discusses key non functional requirements such as availability and scalability. Special attention is given to how the system will handle sudden growth, for example if the company adds fifty thousand new contracts after an acquisition that the system needs to be able to handle that loads effortlessly and efficiently.

1. Architectural Framework Strategy

After comparing different options, TOGAF was chosen as the main architecture framework for this project. It was selected over Zachman because it provides a clear step by step process called the Architecture Development Method which makes it a lot more practical for guiding a full system transformation like the one needed at TechMove. While Zachman is useful for organising ideas, TOGAF is better for actually planning and building the solution.

The first four phases of the ADM will be used to guide the early stages of development.

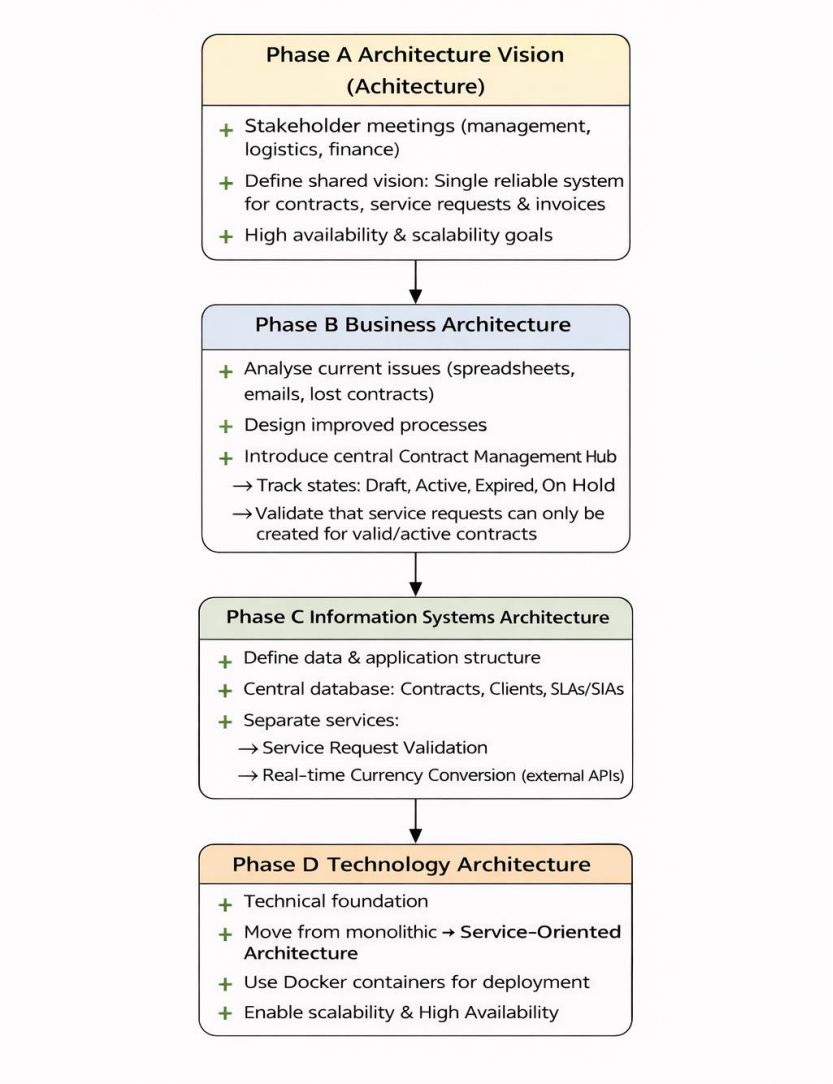
In Phase A, meetings will be held with key stakeholders such as management, logistics teams, and finance staff with a goal to hopefully have a shared vision and create a system for managing contracts, service requests, and invoices, with high availability and enough room for future growth and too keep up with scale.

Phase B, focuses on the current problems and ways the improve the design. Instead of using a handful of tools, the new system will be a Hub. This hub will track contract states such as Draft, Active, Expired, or On Hold, and ensure that service requests can only be created for valid contracts leading to less mistakes.

Phase C, Information Systems Architecture, defines how data and application services will be structured. A database will store every contract, client information, and service agreement. At the same time, separate services will handle tasks like validating service requests and performing real time currency conversions using external APIs all at once meaning less redundancy and less headaches.

Phase D, focuses on the technical setup. The system will move away from a monolithic structure and instead use a service based approach with container technology like Docker. This will make it

easier to scale the system and ensure high availability.



These phases provide a strong starting point for development. Later phases will focus on implementation and long term management, but the early stages are enough to begin designing and building the system.

2. Design Pattern Selection

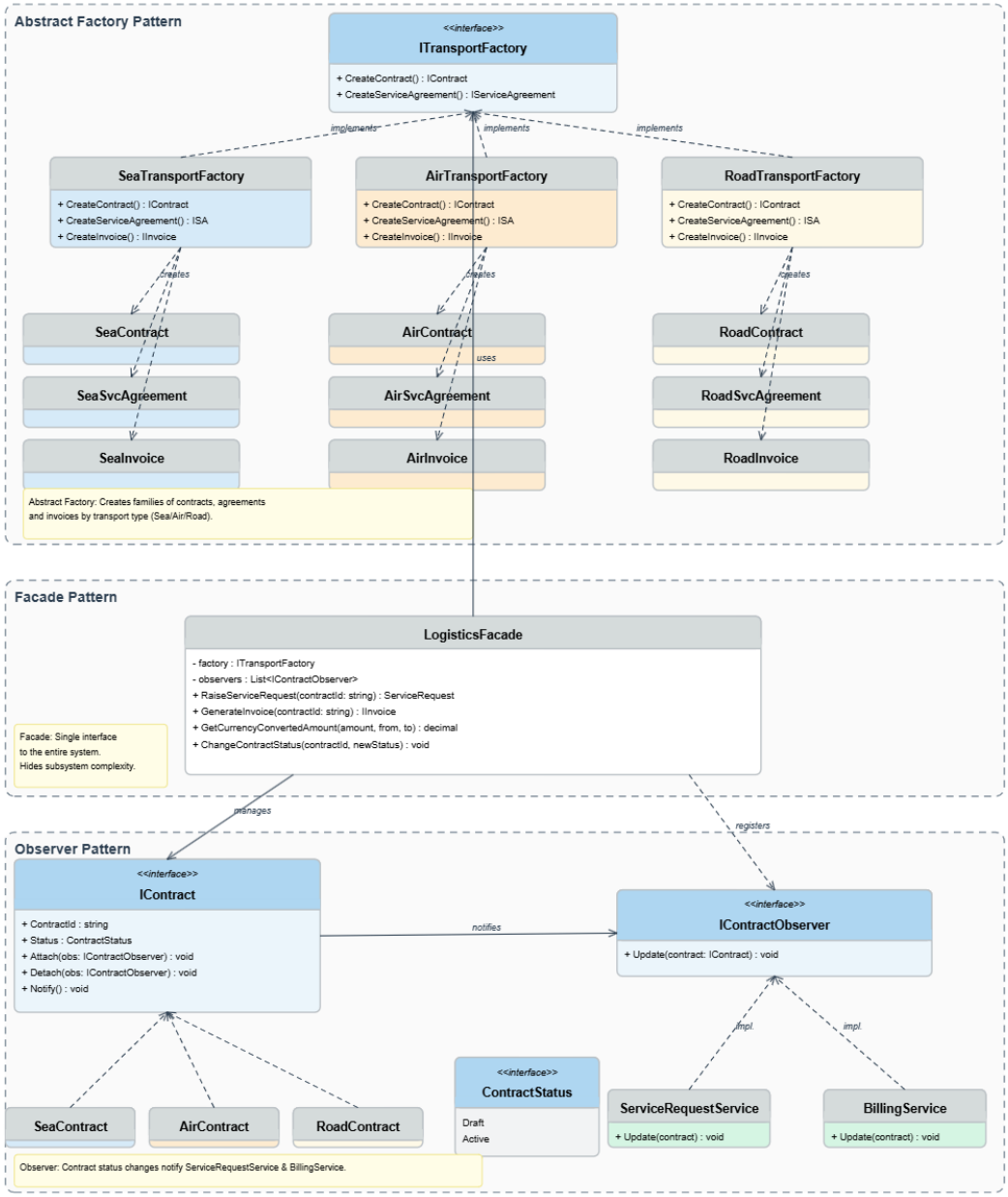
To make sure the system is easy to maintain and expand in the future, three design patterns from the Gang of Four have been selected. These patterns were chosen because they solve real problems in a system like this.

The first pattern is the Abstract Factory pattern. This is used to create groups of related objects without depending on specific classes. In the GLMS, it will be used to create different types of contracts, service agreements, and invoices depending on the transport type such as sea, air, or road. This makes it easy to add new transport types later without changing existing code.

The second pattern is the Facade pattern. This pattern provides a simple interface to a more complex system making everything much more digestible leading to easier understanding. Instead of interacting with multiple subsystems, users will work through a single class. This class will handle communication with the contract system, service request processing, and financial modules. This reduces complexity and makes the system much easier and faster.

The third pattern is the Observer pattern. This pattern allows different parts of the system to automatically react to changes. For example, if a contract changes from Active to Expired, other parts of the system such as service requests or billing will be notified immediately. This helps keep the system consistent without tightly linking different components together.

These patterns will improve the overall structure of the system by reducing dependencies and making the code easier to test and extend..



3. Non Functional Requirements and Scalability

Easily the two most important non functional requirements for this system is availability and interoperability.

The system needs to achieve 99.99% uptime. This means downtime should be very limited over a full year. In logistics even small delays can lead to missed deliveries or financial penalties, so reliability is very important. To support this the system will use techniques such as automatic failover, system monitoring, and container based deployment.

Interoperability is also important because the system needs to communicate with external services. This will be done using APIs, JSON data formats, and secure authentication methods . For example: A currency conversion will use a API to get real time exchange rates to ensure accuracy .

Scalability is critical, especially if the company grows quickly. If thousands of new contracts are added at once, the system must still perform well. Horizontal scaling will be used. This means adding more instances of services instead of upgrading a single machine.

A load balancer will distribute requests across multiple service instances , this will prevent the system from overloading when there is too much traffic going on , dividing that traffic between services. Caching tools will store frequently used data to reduce database load. The system will use Docker containers, and tools that can automatically increase or decrease the number of running services depending on demand.

The database will start with SQLite (as indicated by my lecturer) during development because it is simple and easy to use, but it will later move to a more scalable database with support for replication and distribution.

Other strategies : using techniques like circuit breakers to handle failures , so the systems don't completely destroy themselves if something went wrong just as a extra precaution becuase with systems like there , you can never be too safe. These decisions help ensure the system remains stable even during high usage.

Conclusion

The proposed Global Logistics Management System provides a clear solution to the problems currently faced by TechMove . By using TOGAF and applying suitable design patterns, will make TechMove so much more reliable, scalable, and easier to manage leading to less headaches and delays in operations. I'm positive they will be very pleased once everything is completed.

Moving to a modern all in one service based platform will reduce errors, improve efficiency, and support future growth. Based on this, it is recommended that the project moves forward to the next stage. A.K.A Part 2 .

Reference list

Avolution. (2025). *Best 5 Enterprise Architecture Frameworks | Avolution*. [online] Available at: https://www.avolutionsoftware.com/our-resources/best-enterprise-architecture-frameworks/.

GeeksforGeeks (2020). *Functional vs. Non Functional Requirements*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/software-engineering/functional-vs-non-functional-requirements/.

Visual-paradigm.com. (2019). *Step-by-Step Enterprise Architecture Tutorial with TOGAF*. [online] Available at: https://www.visual-paradigm.com/guide/enterprise-architecture/step-by-step-enterprise-architecture-tutorial-with-togaf-adm/.